

Browsing Large Graphs with Tile Pyramids and Sleeve Routing in the Browser

Anonymous author(s)

Anonymous affiliation(s)

Abstract

We present a new way to visualize a large graph in the style of online geographic maps. The method builds a *tile pyramid* for *semantic zoom*: at every zoom level the labels of the highest-ranked nodes remain readable, just as the names of major geographical features stay readable on those maps.

The edges are routed by a method we call *sleeve routing*, which searches the dual graph of a Constrained Delaunay Triangulation to select a sequence of triangles through the free space, then applies the funnel algorithm to compute a shortest path inside the selected sleeve. We apply several heuristics to speed up the routing.

We implemented our approach in the WebGL renderer of GJS¹, an open-source TypeScript library for graph visualization in web browsers, with the entire pipeline running client-side, without a dedicated server. Our benchmark suite contains nine graphs with up to 32,768 nodes and 236,978 edges, and measures browser-side parsing, layout, routing, and tile-pyramid construction. The renderer's demo can be seen at <https://gjs-sleeve.surge.sh/>.

GJS is available as open source² and as NPM packages³.

2012 ACM Subject Classification Human-centered computing → Graph drawings; Theory of computation → Computational geometry

Keywords and phrases Graph visualization, edge routing, Constrained Delaunay Triangulation, tiling, WebGL

Category Track 2, Long Paper

Generative AI Declaration Generative AI was used throughout the preparation of this article and the accompanying software, including writing and refactoring source code, generating figures, drafting and editing technical prose, and running tests and benchmarks. The authors reviewed all AI-assisted content and take responsibility for the final form.

1 Introduction

Visualizing graphs in web browsers without a dedicated server presents unique challenges: the runtime is single-threaded, the per-tab JavaScript heap is capped at roughly 4 GB,⁴ and users expect the smooth pan-and-zoom experience familiar from online maps. GJS is an open-source TypeScript library that addresses these challenges by combining efficient layout algorithms, a novel edge routing method, and a tiling scheme with semantic zoom.

GJS is consumed as a set of NPM packages and renders graphs using WebGL through the deck.gl framework [2]. It supports the layered Sugiyama scheme [46], Pivot MDS [15], and IPSep-CoLa [22], with edges routed around node obstacles. Throughout this paper our typical example is a social network laid out by IPSep-CoLa; the methods we describe, however, are agnostic to the layout algorithm and apply to any node placement in which the

¹ GJS is a placeholder name used during double-blind review; the actual library name will be disclosed in the camera-ready version.

² Anonymized source code mirror for review: <https://anonymous.4open.science/r/gjs-163F/>

³ Package names omitted for double-blind review.

⁴ In current Chrome, V8 with pointer compression caps the per-tab JavaScript heap at approximately 4 GB (2^{32} bytes).



© Anonymous author(s);

licensed under Creative Commons License CC-BY 4.0

34th International Symposium on Graph Drawing and Network Visualization (GD 2026).

Editors: TBD; Article No. 0; pp. 0:1–0:15



Leibniz International Proceedings in Informatics

LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

nodes do not overlap. The graph loading pipeline builds a hierarchy of tiles—analogueous to map tiles—so that at any zoom level only a bounded number of entities are drawn. Tiles are delivered to the browser through the standard `TileLayer` of the `@deck.gl/geo-layers` package, giving smooth pan and zoom. A live demo of the WebGL renderer is available online⁵.

This paper makes two main contributions:

1. A *tiling scheme* for large graph visualization that builds a hierarchical tile pyramid, limits visible entities per tile using PageRank-based filtering, simplifies edge routes at coarser levels, and integrates with the WebGL renderer for real-time pan and zoom.
2. A *sleeve routing* algorithm that routes edges on the dual graph of a Constrained Delaunay Triangulation, using per-source batched Dijkstra trees followed by the classical funnel algorithm to produce shortest-path geodesics in homotopy classes.

2 Related Work

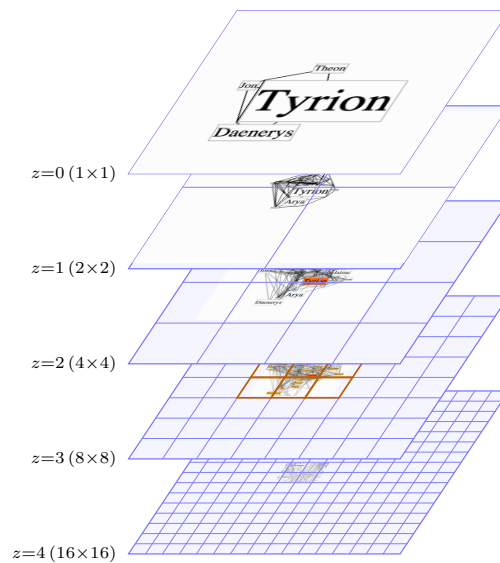
2.0.0.1 Graph visualization tools.

Graphviz [26, 5] is a long-standing graph drawing tool supporting multiple layout algorithms, including the Sugiyama method and Scalable Force-Directed Placement [7]; it does not support tiling or WebGL rendering. MSAGL [41] is a desktop layout engine that did not have the features described here. On the web, Sigma.js [8] and Cytoscape.js [24] provide interactive graph rendering, but rely on straight-line or generic spline edges without obstacle-avoiding routing and without a pyramid of precomputed tiles. ReGraph [6] uses WebGL to render large graphs with straight-line edges and supports interactive cluster expansion but not tiling. Cosmograph [1] uses GPU-accelerated force-directed layout for million-node graphs with straight-line edges, without tiling. GraphMaps [40] introduced the tile-pyramid and node-ranking scheme, but it runs only on Windows, ships raster tiles, and reuses one global edge routing that is merely clipped per tile; GJS brings the idea to the browser with vector tiles and per-level sleeve routing. Cornac [43] handles huge graphs with tiles and edge bundling [35, 33] but requires a multi-server backend. Earlier navigation schemes for large graphs include ASK-GraphView [12], which organizes nodes into a hierarchical cluster tree, and topological fisheye views [25], which distort a multilevel layout [28, 34] under the focus; neither targets browser-side tile pyramids or per-level edge routing. Hierarchical edge bundling [32] and force-directed bundling [33] are complementary clutter-reduction techniques applied *after* edges have been routed.

2.0.0.2 Shortest paths and edge routing.

Our sleeve algorithm walks the dual of a Constrained Delaunay Triangulation [45] and relies on the classical funnel algorithm for extracting a geodesic from a triangle strip [38, 16, 27, 4]. The optimal algorithm for Euclidean shortest paths among polygonal obstacles [31] and the recent $O(m \log^{2/3} n)$ single-source shortest-path algorithm that breaks the sorting barrier [21] are remarkable theoretical results, but they are not practical: both build elaborate global data structures that we avoid. Graphviz builds the full visibility graph for edge routing in force-directed layouts, which has $O(n^2)$ edges. For Sugiyama layouts, it uses the funnel algorithm [19] but requires a simple containing polygon. yWorks [11] offers “Organic edge routing” based on force-directed simulation. The approach of Dwyer and Nachmanson [23]

⁵ <https://gjs-sleeve.surge.sh/>



121 ■ **Figure 1** The entities rendered on each level are shown without scaling as the user zooms in.

71 routes on a Yao-graph spanner with local shortcutting. For orthogonal layouts, the libavoid
 72 framework of Wybrow, Marriott and Stuckey performs obstacle-avoiding connector routing
 73 with incremental updates [47].

74 2.0.0.3 CDT-based pathfinding.

75 The closest precedent to our routing pipeline is the CDT-and-funnel pathfinder of Demyen
 76 and Buro [18], originally designed for game-AI units of nonzero radius. They run admissible
 77 A* search on the CDT dual (with a reduced-graph variant that collapses corridor chains into
 78 single abstract edges) to recover provably shortest paths. Their search is, however, per-query:
 79 the bounds depend on the start and goal, so it cannot share work across the edges with the
 80 same source as our per-source Dijkstra does. For the graphs we target this would be roughly
 81 an order of magnitude slower; we therefore trade their proven optimality for speed.

82 3 Tiling

83 The input to tiling is a laid-out graph together with routed edges: we prefer to route the
 84 edges with sleeve routing, Section 4, to achieve visual stability across level changes. The
 85 output is a sequence of $Z+1$ levels: level Z is the finest, and level 0 is the coarsest. Each
 86 level is a collection of tiles indexed by integer (x, y) grid coordinates.

87 A tile is a rectangular viewport together with its *tile data*: nodes, edge labels, edge
 88 arrowheads, and *edge clips*. An edge clip is a curve confined to the tile rectangle, paired with
 89 the non-empty list of graph edges whose routes enter and exit the tile at very close endpoints.
 90 Bundles usually capture the stretches on which several routes travel close together inside a
 91 common sleeve; we represent each such stretch as a single clip with an attached edge list,
 92 rather than as one clip per edge.

120 Pseudocode 1 describes the algorithm building the tile pyramid.

93 **Algorithm 1** BUILD_TILE_PYRAMID($G, \text{TileCapacity}, \text{minTileSize}, \mathcal{M}_{\max}$)

```

94 1:  $T_0 \leftarrow$  single tile whose rectangle is the smallest axis-aligned square of side  $2^{\lceil \log_2 \max(w,h) \rceil}$ 
95    centered on  $G$ 's bounding box of width  $w$  and height  $h$ , holding all of  $G$ 
96 2:  $\text{levels} \leftarrow [T_0]$ ;  $z \leftarrow 0$ ;  $\text{used} \leftarrow |T_0|$   $\triangleright |T| = \text{element count of tile } T$ 
97 3: while some tile in  $\text{levels}[z]$  exceeds  $\text{TileCapacity}$  elements do
98 4:    $\text{next} \leftarrow$  empty  $2^{z+1} \times 2^{z+1}$  tile grid
99 5:   for all tiles  $T \in \text{levels}[z]$  do  $\triangleright$  per-tile inner loop
100 6:     split  $T$  into its four sub-tiles of  $\text{next}$   $\triangleright$  Section 3.2
101 7:      $\text{used} +=$  number of new elements added to  $\text{next}$ 
102 8:     if  $200 \cdot \text{used} > \mathcal{M}_{\max}$  then  $\triangleright$  memory budget; checked mid-split
103 9:       discard  $\text{next}$  and break pyramid growth
104 10:    end if
105 11:  end for
106 12:  if  $\text{next}$ 's tile width or height is below  $\text{minTileSize}$  then
107 13:    break  $\triangleright$  tile size below threshold
108 14:  end if
109 15:   $\text{levels}[z+1] \leftarrow \text{next}$ ;  $z \leftarrow z + 1$ 
110 16: end while
111 17:  $Z \leftarrow z$ ;  $V_Z \leftarrow V(G)$ ;  $s_v^{(Z)} \leftarrow 1$  for all  $v \in V_Z$ 
112 18:  $V_{\text{PR}} \leftarrow$  nodes of  $G$  sorted by descending PageRank
113 19: for  $z = Z - 1$  down to 0 do
114 20:    $(V_z, \{s_v^{(z)}\}) \leftarrow \text{SELECT\_TOP\_K\_WITH\_ADAPTIVE\_SCALE}(V_{\text{PR}}, \lceil |V|/2^{Z-z} \rceil)$   $\triangleright$  Section 3.1
115 21:   build a CDT from the inflated polylines of  $V_z$  at scales  $\{s_v^{(z)}\}$ 
116 22:   route every edge  $e \in E_z$  on this CDT
117 23:   rebuild the tiles of  $\text{levels}[z]$   $\triangleright$  Section 3.2
118 24: end for
119 25: return  $\text{levels}$ 

```

122 3.0.0.1 How the index of the finest layer, Z , is chosen.

123 We treat the tiles differently when calculating Z and when preparing for rendering. There is
 124 no element filtering in the former case: all graph entities are represented in each level. We
 125 continue increasing Z , generating new levels, and split tiles, until a stopping condition is
 126 met. Then, while keeping the finest level intact, we rebuild the tiles of levels $0, \dots, Z - 1$ for
 127 rendering.

128 We start from $Z = 0$ and grow the pyramid one level at a time. We stop as soon as any
 129 of three conditions is met:

- 130 1. every tile at the new level contains at most $\mathcal{C}$ elements, with default $\mathcal{C} = 500$;
- 131 2. the new tile width and height are both below ten times the average node width and
 132 height;
- 133 3. while generating the tiles of the new finest level, the running total of stored tile elements
 134 summed over all levels, multiplied by 200 bytes per element, exceeds the memory budget
 135 $\mathcal{M}_{\max}$, with default 4 GB; the partial level is then discarded and Z is set to the previous
 136 finest level.

137 The third condition is checked incrementally inside the level-build loop, so we abort as soon
 138 as the memory barrier is reached. On every benchmark graph in Table 1 the first condition
 139 fires; the second and third are defensive guards that bound depth on hypothetical inputs
 140 whose finest level cannot be packed within $\mathcal{C} = 500$.

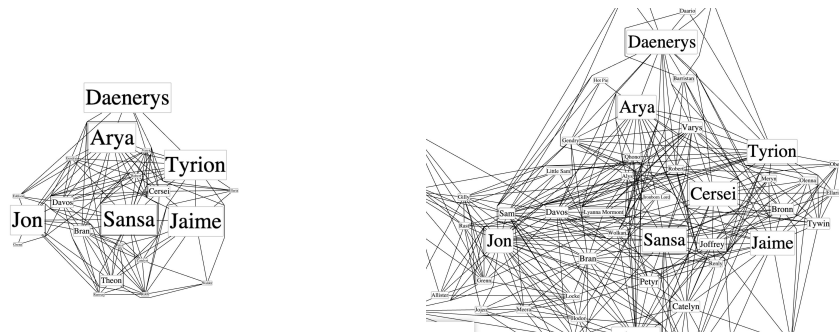


Figure 2 Two adjacent pyramid levels of the Game of Thrones graph rendered by the WebGL renderer of GJS with the sleeve router. The coarser level is on the left, the finer one on the right; both panels are cropped to the same world window. For each level a new graph is assembled from a PageRank-ranked prefix of the nodes, and edges are routed against the obstacles introduced at that level.

3.0.0.2 Three phases.

Once Z and the tile rectangles are fixed, each coarser level $z = Z-1, Z-2, \dots, 0$ is built in three phases:

1. pick the nodes of level z with PageRank and possibly enlarge them, Section 3.1;
2. reroute the edges of G_z around the resulting obstacles, Section 4;
3. recompute the per-tile edge clips on level z , Section 3.2.

Figure 2 shows two adjacent pyramid levels built by this procedure; Algorithm 1 summarizes the whole pipeline.

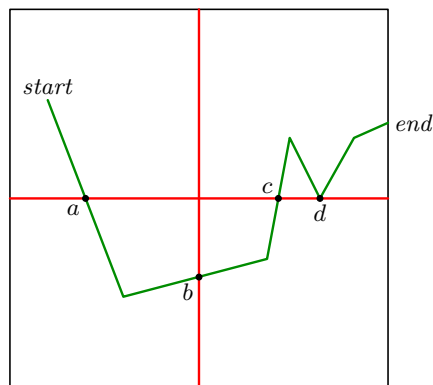
3.1 PageRank-Guided Level Construction

Before filling the level's tiles for rendering we run PageRank [42] on G once, and sort all nodes of the graph, V , by descending rank. Write $k = Z - z$ for the level's depth below the finest level. The candidates for level of depth k are the nodes of the *prefix* of the sorted array of length $\lceil |V|/2^k \rceil$. The level's edge set, used by Algorithm 1, is the subset of $E(G)$ whose endpoints both lie in this prefix.

Node positions are inherited from the original layout and never move: across levels we change only each node's display size.

We add nodes to a level of depth k by walking the level's prefix in rank-descending order. The top-ranked node is scaled by 2^k . We keep the node scales non-increasing, but as large as possible, while making sure that each next node does not overlap the previously selected nodes. If even at its original size the candidate's bounding box would overlap an already-accepted node's scaled box, the candidate is dropped.

Dropping a relatively high-ranked candidate at a coarse level is a deliberate compromise: we trade the inclusion of that candidate at this level for the invariant that the rendered nodes at every level remain pairwise disjoint. The cost is small in practice. Consider Arya's node in the *Game of Thrones* graph: as Figure 1 shows, it is dropped from the top layer because a higher-ranked node, Tyrion, has covered the available region. The user still sees that part of the graph as occupied, and Arya's individual node appears at a finer level where its box fits.



197 **Figure 3** An edge clip, drawn as a polyline, from *start* to *end* inside a tile. The two red midlines
 198 split the tile into four sub-tiles and meet the polyline at the four points *a*, *b*, *c*, *d*; at *d* the polyline
 199 touches the horizontal midline without crossing it. Cutting the polyline at these points yields
 200 five sub-clips $[start, a]$, $[a, b]$, $[b, c]$, $[c, d]$, $[d, end]$, each contained in one sub-tile and meeting that
 201 sub-tile's boundary only at its endpoints.

173 3.1.0.1 Spatial-hash overlap test.

174 A naive overlap test against every previously accepted node would dominate the cost of the
 175 filter. We instead maintain the accepted boxes in a uniform spatial hash whose cell size is
 176 chosen so that any accepted box, and any candidate's box at the level's maximum scale,
 177 both fit inside one cell on each axis. Because of this choice, two such boxes can overlap only
 178 if their center cells differ by at most one step on each axis: testing a candidate therefore
 179 reduces to scanning the accepted boxes registered in the candidate's own cell and in the eight
 180 cells immediately around it.

181 The hash brings little benefit at the coarsest levels, where the prefix is short and even
 182 a naive scan over the few accepted boxes is cheap; it pays off at finer levels, where the
 183 prefix grows geometrically and a per-candidate constant-time test replaces a linear scan
 184 over hundreds of already-accepted boxes. On the layouts we tested the expected work per
 185 candidate is constant, so the overall cost is dominated by the initial sort by PageRank. In
 186 our experiments this phase was never a bottleneck: the per-level routing dominates the build
 187 time at every level we measured.

188 3.2 Splitting Edges between Tiles

189 We speed up edge-clip generation as follows. Let the *midlines* of a tile be the horizontal and
 190 vertical segments through its center; they split the tile into four sub-tiles. Consider a tile and
 191 an edge clip whose curve meets the tile boundary only at the curve's endpoints. Cutting the
 192 curve with the tile's midlines yields a set of sub-clips, each contained in one sub-tile, and each
 193 preserves the invariant: its curve meets the sub-tile boundary only at its endpoints. Because
 194 of this invariant, every split reuses the parent clip's two boundary crossings, and only the
 195 curve-vs-midline intersections need to be computed; no full curve-vs-rectangle clipping is
 196 required. Figure 3 illustrates one such split.

202 When we grow the pyramid in line 6 of Algorithm 1, each edge clip is already contained
 203 in its parent tile, so subdivision starts from that parent. When we prepare the tiles for
 204 rendering in line 23 of Algorithm 1, we have the rerouted graph G_z , so for each edge of G_z
 205 we start the subdivision from an initially empty temporary tile whose extent covers the whole

Table 1 Maximum number of elements rendered in a single tile across the whole pyramid for several graphs (capacity $\mathcal{C} = 500$; Z chosen by the natural stops of Section 3). The capacity $\mathcal{C}$ only guides the choice of Z ; we have no tight analytical bound on the actual rendered per-tile element count, which is therefore reported empirically. “Max tile’s level” is the level on which the maximum tile was found. In the worst cases, **ca-HepPh** and **ca-CondMat**, the densest tile holds $6\text{--}8\times$ the nominal capacity $\mathcal{C}$.

Graph	$ V $	$ E $	Levels built	Max per tile	Max tile’s level
gameofthrones	407	2 639	5	324	4
composers	3 405	13 832	8	749	3
ca-GrQc	5 242	28 968	8	359	4
ca-HepTh	9 877	51 946	8	1 836	3
facebook_combined	4 039	88 234	9	928	5
ca-HepPh	12 008	236 978	9	3 643	3
ca-CondMat	23 133	186 878	8	2 949	3
deezer_europe	28 281	92 752	8	2 402	3
delaunay_n15	32 768	98 274	8	283	7

graph.

3.2.0.1 Bundling edge clips.

To correctly reflect the number of elements in a tile and further reduce the number of elements in a tile, all clips within a tile that share the same, or almost the same, pair of endpoints are bundled into a single clip whose attached edge list accumulates every contributing edge; the bundled geometry is taken from the first such clip encountered during splitting, and the choice is consistent within each level. Table 1 reports the resulting maximum number of elements rendered in a single tile across the entire pyramid for a range of graphs.

4 Sleeve Routing on the CDT

To prepare for routing given the graph layout, each node is covered by a padded obstacle polygon. We compute a Constrained Delaunay Triangulation $\mathcal{T}$ on the set of these polygons [17, 20]. The *dual graph* D of $\mathcal{T}$ has one node per triangle and one edge for each pair of triangles sharing a side. The weight of an edge is the Euclidean distance between the centroids of its triangles.

4.1 Routing inside a Sleeve

We split routing an edge into two subproblems: a *combinatorial* one—which sequence of diagonals to cross—and a *geometric* one—which exact polyline through that sequence is shortest. The geometric step is the classical funnel algorithm [16, 30], which pulls the path taut inside the polygon formed by the sleeve triangles. The CDT alone is already a constant-stretch spanner of the visibility graph of its constraint set: the shortest path along CDT edges between two mutually visible vertices is at most $4\pi\sqrt{3}/9 \approx 2.42$ times the visibility-geodesic distance [14]. Funnel optimization on the sleeve can only further shorten the path, and returns the visibility-geodesic exactly whenever the chosen sleeve contains it.

To obtain the sleeve we run a shortest-path search on D , where triangles inside obstacles other than the source and target obstacles are excluded. The natural per-edge instantiation

is A* [29] with the straight-line distance to t as heuristic. When a node has many incident edges, however, running independent searches is wasteful. We instead group edges by source s and run a single Dijkstra computation per source, expanding until all targets are reached and recovering each sleeve by following parent pointers. This gives a speedup over A*; for example on Game of Thrones, with 407 nodes and 2 639 edges, it cuts routing time by $\sim 23\%$. Table 3 reports the gain across the full benchmark suite.

We can improve even more. Since the query edges are undirected on D , each can be served from either endpoint, so the number of distinct roots is in turn reducible by reorienting query edges to minimize that count. Let the *demand graph* H be the graph whose vertices are the graph nodes and whose edges are the query edges still to be routed. A set of roots R is feasible iff every demand edge has at least one endpoint in R , that is, R is a vertex cover of H , and the smallest such R is a minimum vertex cover of H . The problem of finding a minimum vertex cover is NP-hard [36], so we approximate it with the standard greedy maximum-degree rule: while H still has an edge, pick a vertex of maximum degree in H , add it to R , and delete it together with its incident edges. Implemented with degree buckets the procedure runs in linear time. Each demand edge is then routed from whichever endpoint lies in R . To our knowledge no prior edge router exploits a vertex cover of the demand graph in this way; many-to-many shortest-path frameworks [37] also amortize per-source searches, but they take the source set as given and do not reduce it through a covering argument. Applying this heuristic reduces the number of Dijkstra trees on Game of Thrones from 331, one per distinct source, to 198, and on the composers graph with 3 405 nodes and 13 832 edges from 2 645 to 1 281, shortening the routing pass by $\sim 28\%$ and $\sim 31\%$ respectively.

In addition, before launching the sleeve search for an edge we try routing the straight line segment between the source and the target by walking the CDT.

4.1.0.1 Edge hint.

We explored a routing variant in which the path generated for an edge on one level provides a hint for routing the same edge on an adjacent level, increasing the visual stability of the routes between levels. Its shortcoming is that the search cannot be batched per source as the Dijkstra-tree variant above is, which impacts performance.

4.2 Vertex Collapse at Source and Target

We improve the sleeve’s geometry in the situation depicted in Figure 4. The remedy is to *collapse* corner-hugging chain vertices to the corresponding endpoint. Walking the right chain of the sleeve outward from s , we find the first vertex of the source obstacle at which the chain makes a right turn, and replace every source-owned vertex on that chain up to and including this one by s . The left chain is processed mirror-symmetrically, with left turns instead of right, and the target end likewise. Diagonals whose two endpoints coincide after collapse are dropped, and the left/right orientation of each surviving diagonal is preserved from the uncollapsed geometry. The collapsed vertex in Figure 4(a) is marked in yellow.

After collapse, the funnel sees a wide opening at each endpoint, as in Figure 4(b), and naturally selects the optimal exit direction. A final arrowhead-trimming step clips the path at the node boundary curves. Collapse might cause the resulting path to overlap nodes other than the source and the target, but we have found the effect to be negligible.

The implementation also offers an optional pass that fits a Bezier segment into each polyline corner to render edges as smooth curves. It is disabled by default for performance:

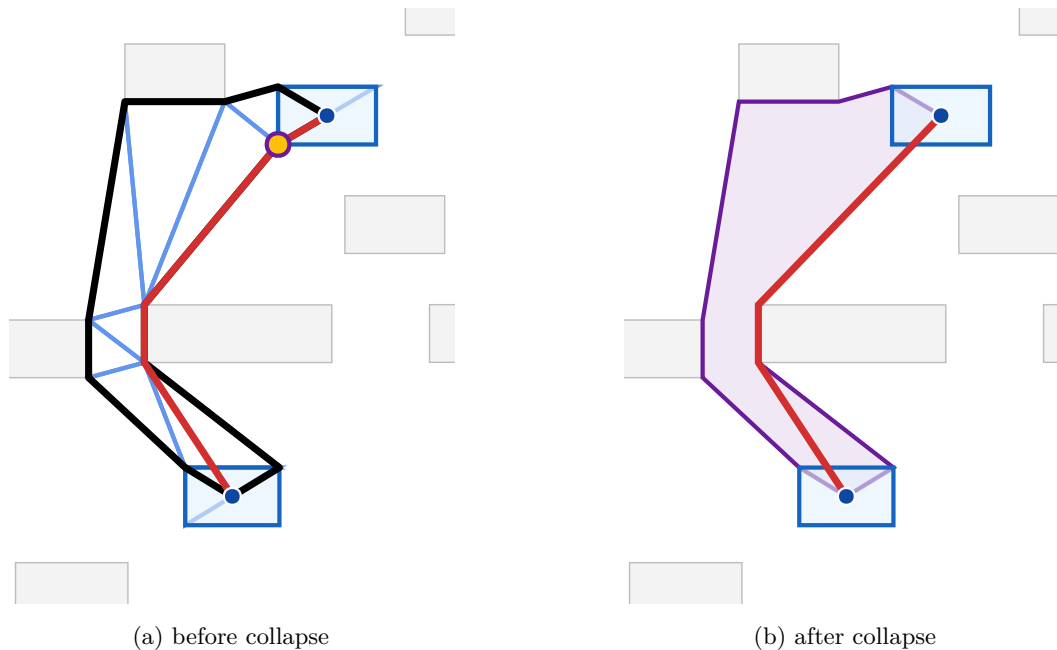


Figure 4 Effect of vertex collapse on the LORAS-RENLY edge of the Game of Thrones graph. Endpoints are shown as blue dots; their padded obstacles are highlighted in light blue. (a) The CDT triangles forming the sleeve are drawn in cornflower blue and the funnel path in red; the thick black polyline is the perimeter of the non-collapsed sleeve. The yellow dot marks the source-obstacle vertex that gets collapsed. (b) The sleeve after collapse, drawn as a single purple polygon with the source/target obstacles merged into the corresponding endpoints, and the funnel path (red) through it. Collapsing widens the funnel opening at the endpoint and removes the spurious detour around the obstacle corner.

the polyline rendering is fast and visually acceptable, so we trade the smoother curves for routing speed.

5 Rendering with deck.gl

The output of the build pipeline of Section 3 is consumed in the browser by the `TileLayer` of `@deck.gl/geo-layers` [10]. `TileLayer` is the standard component used by online tile-map renderers: it tracks the viewport, determines which tiles are needed at the current zoom, and asynchronously requests them through a user-supplied `getTileData` callback, caching the results across frames so that subsequent pans within the cache require no further work.

GJS integrates with this component at construction time by exposing the precomputed pyramid as the `TileLayer`'s zoom range, with the coarsest pyramid level mapped to the layer's lowest zoom and the finest level to the highest. As the user pans and zooms, `TileLayer` resolves the viewport to an integer zoom and a set of axis-aligned tile indices and, for each tile it needs to display, calls `getTileData`. The renderer translates each such request back into a tile of the corresponding pyramid level and returns its precomputed nodes, edge clips, edge labels, and arrowheads. The mapping is one-to-one, so no geometric work is performed at view time: panning shifts the visible window over the tile cache, and zooming displays tiles from the adjacent level using `TileLayer`'s built-in cross-fade.

The rendered geometry depends only on the zoom level and does not change with the

Graph	$ V $	$ E $	Routing	Tiling	Total
gameofthrones	407	2 639	0.07	0.07	0.38
composers	3 405	13 832	1.49	0.74	2.69
ca-GrQc	5 242	28 968	1.78	0.87	3.40
facebook_combined	4 039	88 234	4.34	2.36	7.77
ca-HepTh	9 877	51 946	9.33	3.41	14.87
ca-HepPh	12 008	236 978	38.89	14.08	57.71
ca-CondMat	23 133	186 878	79.72	20.52	110.06
deezer_europe	28 281	92 752	134.78	23.09	172.68
delaunay_n15	32 768	98 274	18.71	5.64	41.38

Table 2 End-to-end loading benchmarks in headless Chrome 148 on an Apple M4 Max laptop, all times in seconds. Measurements come from the `chrome-routing-bench` example, which runs the unmodified browser bundle of GJS. *Routing* sums the CDT-construction and Dijkstra-tree sleeve-search phases of Section 4.1 on the full graph; *Tiling* is the build of the tile pyramid, capacity $C = 500$ with Z chosen by the natural stops of Section 3, which includes the per-level routing; *Total* is the total loading time including parsing of the graph file, graph layout, and all other stages.

viewport, except for the clipping to the visible window. Within each level, the displayed nodes are the ones chosen for that level by the construction of Section 3.1, and the displayed edges are the geodesics around those nodes, computed by the sleeve router of Section 4.

5.0.0.1 Interaction.

The renderer supports pan, zoom, and hover highlighting of nodes and edges. Hovering an edge highlights it and its incident nodes; hovering a node highlights the node, its one-hop neighbors, and its two-hop neighbors, each in a different color. A `zoomTo` API animates the viewport to a target rectangle with a smooth transition and is used, for example, by the search box to focus the view on a query result.

6 Experimental Results

We evaluate the sleeve router and the tile-pyramid build on the same benchmark graphs as Table 1: the Game of Thrones social network [13], the GD 2011 contest `composers` graph [9], four SNAP collaboration networks (`ca-GrQc`, `ca-HepTh`, `ca-HepPh`, `ca-CondMat`) and the SNAP `facebook_combined` ego network [39, 3], the `deezer_europe` social network [44], and the SuiteSparse Delaunay mesh `delaunay_n15`. All experiments run in headless Chrome 148 on an Apple M4 Max laptop with 48 GB of unified memory, with the unmodified browser bundle of GJS loaded into the page. Layout uses IPSep-CoLa, and the tile pyramid is built with the same parameters as Table 1: capacity $C = 500$, with Z chosen by the natural stops of Section 3.

Table 2 reports per-graph timings of three pipeline stages: *Routing*, the sum of the two phases logged by the sleeve router on the full graph—constrained Delaunay triangulation of the obstacles and the Dijkstra-tree search on the CDT dual, Section 4.1; *Tiling*, the build of the tile pyramid, Section 3, which includes the per-level routing; and *Total*, the end-to-end loading time including parsing.

The CDT phase contributes negligibly to the routing column—from 6 ms on Game of Thrones to about half a second on the largest graphs—so virtually all of *Routing* is the Dijkstra-tree sleeve search, with the vertex-cover heuristic of Section 4.1 reducing the number of trees we run. Even on graphs with hundreds of thousands of edges (`ca-CondMat`, `ca-HepPh`, `deezer_europe`) the full pipeline finishes in a few minutes; this is a one-time build cost. The

Graph	Srcs	VC	Time (s)			Speedup	
			A*	DJ	VC_DJ	DJ/A*	VC/DJ
gameofthrones	331	198	0.077	0.059	0.043	+23 %	+28 %
composers	2 645	1 281	2.60	2.14	1.49	+18 %	+31 %
ca-GrQc	5 241	2 796	2.55	2.52	1.65	+1 %	+35 %
facebook_combined	3 663	3 045	13.31	4.88	4.36	+63 %	+11 %
ca-HepTh	9 875	5 003	17.13	15.42	9.28	+10 %	+40 %
ca-HepPh	12 006	7 032	111.2	58.07	36.38	+48 %	+37 %
ca-CondMat	23 133	13 564	189.1	131.1	76.45	+31 %	+42 %
deezer_europe	21 060	13 450	172.1	154.0	124.4	+11 %	+19 %
delaunay_n15	28 467	23 456	13.57	16.02	14.88	-18 %	+7 %

Table 3 Sleeve-search times of the three routing modes on the benchmark graphs of Table 2, in headless Chrome 148. Columns: *Srcs*, the number of distinct edge endpoints in the demand graph—one per source for the per-edge A* baseline; *VC*, the number of Dijkstra trees actually launched after the greedy maximum-degree vertex-cover reorientation of Section 4.1; *A**, wall-clock seconds of one full sleeve-search pass with per-edge A* (straight-line distance to the target as heuristic); *DJ*, wall-clock seconds with the batched Dijkstra-tree mode—one tree per source, expanding until all of that source’s targets are reached; *VC_DJ*, wall-clock seconds with the vertex-cover-reoriented Dijkstra-tree mode—same as DJ but with only *VC* roots; $DJ/A^* = (A^* - DJ)/A^*$, speedup of DJ over A*; $VC/DJ = (DJ - VC_DJ)/DJ$, additional speedup of VC_DJ over DJ. The CDT-construction phase, identical across modes, is excluded so that only the search costs are compared.

resulting tile pyramid already contains the per-level routes, so panning and zooming only fetch and render the precomputed tiles, with no rerouting at view time.

Table 3 isolates the contribution of the two source-side optimizations of Section 4.1: batched Dijkstra trees over per-edge A* and the vertex-cover heuristic on top of that. Each row times a single sleeve-search pass run after layout, with the constrained Delaunay triangulation excluded so that only the search costs are compared. The batched Dijkstra trees beat per-edge A* on every social graph, with the gain growing with the average degree and reaching +63 % on **facebook_combined** and +48 % on **ca-HepPh**; the vertex-cover heuristic then shaves a further 11–42 % off those same graphs. The Delaunay mesh **delaunay_n15** is the only outlier: its source set is almost in bijection with its vertices, so neither optimization has many opportunities to share paths and the bookkeeping leaves them marginally slower than the baselines.

To assess route quality, we compared the sleeve router with the optimum on the full visibility graph of the padded obstacles, on the Game of Thrones layout. The visibility-graph optimum is the shortest taut polyline around the obstacles; we computed it by all-pairs Dijkstra over the polygon corners and the node centers. This is a lower bound on any obstacle-avoiding route. On the 407-node, 2,639-edge layout the sleeve router has total arc length 714.3 k arc-length units against 695.1 k for the visibility-graph optimum, a ratio of 1.028×. The worst per-edge ratio is 1.37, much less than the constrained Delaunay triangulation’s worst-case stretch $4\pi\sqrt{3}/9 \approx 2.42$ [14].

6.0.0.1 Browsing smoothness.

We test how smoothly a user can browse a freshly loaded graph in the browser. On the same M4 Max laptop as Tables 2 and 3, a script loads a graph, waits for the tile pyramid to build, and then plays back twelve dives. Each dive picks a random point inside the laid-out graph, recenters the view on that point, zooms from the coarsest view all the way in to the finest detail in one second, then zooms back out to the coarsest view in another second. The twelve random points are independent and the dives run back to back.

Graph	$ V $	$ E $	Z	LT	LT total (ms)	Frame p95 (ms)	Elts/frame max
gameofthrones	407	2 639	5	0	0	17.4	2 567
composers	3 405	13 832	9	1	51	18.2	497
ca-GrQc	5 242	28 968	9	0	0	18.2	677
ca-HepTh	9 877	51 946	9	0	0	18.2	383
facebook_combined	4 039	88 234	9	0	0	18.0	495
ca-HepPh	12 008	236 978	9	9	556	18.4	1 394
ca-CondMat	23 133	186 878	9	5	273	18.2	1 391
deezer_europe	28 281	92 752	9	3	164	18.4	2 505
delaunay_n15	32 768	98 274	9	0	0	18.1	1 012

Table 4 Browsing-smoothness measurements on the benchmark graphs. For each graph the script plays back twelve dives; each dive picks a random point inside the laid-out graph, recenters on it, and zooms from the coarsest view to the finest and back in two seconds. Z , number of tile-pyramid levels built; LT , number of main-thread pauses longer than 50 ms observed during the twelve dives; LT total, the total duration of those pauses, in milliseconds; *Frame p95*, the 95th percentile of the gap between successive on-screen refreshes during the twelve dives, in milliseconds; *Elts/frame max*, the largest number of graphical objects, that is, nodes, edge clips, labels, and arrowheads, that the browser had to draw at the deepest point of any dive.

While the dives play, the script measures three things on the browser side. First, how often the screen actually refreshes: at 60 Hz the screen refreshes every 16.7 ms, and a longer gap is visible as a stutter. We report the 95th percentile of the gap between successive refreshes, the value below which 95 % of refreshes fall. Second, how often the browser pauses for more than 50 ms on a single block of work; this is the standard browser metric of a “long task” and each one freezes the page. Third, at the deepest point of every dive, how many graphical objects the browser must draw, that is, nodes, edge clips, labels, and arrowheads, summed over the tiles that fall inside the viewport. This last number is a proxy for how hard the GPU has to work each frame. Table 4 reports the result.

Across all nine graphs the dives stay close to the 60 Hz target. On every graph, 95 % of the gaps between successive refreshes are at most 18.4 ms, only 1.7 ms over the ideal 16.7 ms gap at 60 Hz and too small to be perceived as a stutter. The browser pauses for more than 50 ms only a handful of times across the twelve dives on the three densest graphs, *ca-HepPh*, *ca-CondMat*, and *deezer_europe*, and not at all on the smaller ones, so the moments of stutter that do happen are short and rare. At the deepest zoom, the browser is asked to draw at most 2 567 objects on *gameofthrones* and at most 2 505 on *deezer_europe* even though the latter graph has 35 times more edges. This is the property the pyramid is designed to deliver: how hard the GPU has to work each frame depends on what is currently on screen, not on the total size of the graph.

7 Conclusion

We presented GJS, an open-source TypeScript library for graph layout, edge routing, and visualization in web browsers. The sleeve routing method routes edges directly on the CDT dual graph using batched Dijkstra trees and the funnel algorithm. The tiling scheme enables map-like exploration of large graphs with level-of-detail filtering and edge simplification. The WebGL renderer, built on *deck.gl*, provides smooth pan-and-zoom interaction.

7.0.0.1 Future work.

The current approach builds the entire tile pyramid up front, which is expensive and bounds the size of graphs we can handle. A natural next step is to build only the first few levels

quickly and hand them to the renderer; the remaining levels would be built lazily as the user browses the graph. This would lower the time-to-first-frame and lift the limit on graph size.

A second challenge is that some tiles can contain many more elements than the capacity parameter $\mathcal{C}$ that guides the build. Table 1 reports densest tiles holding $6\text{--}8\times$ the nominal $\mathcal{C}$ in the worst cases, which leaves the rendering and interaction cost uneven across the viewport. Enforcing an actual per-tile budget remains an open problem.

References

- 1 Cosmograph. <https://cosmograph.app>.
- 2 deck.gl: Large-scale WebGL-powered data visualization. <https://deck.gl/>.
- 3 facebookcombined. https://snap.stanford.edu/data/facebook_combined.txt.gz.
- 4 Funnel algorithm. <https://page.mi.fu-berlin.de/mulzer/notes/alggeo/polySP.pdf>.
- 5 Graphviz. <http://www.graphviz.org/>.
- 6 Regraph. <https://cambridge-intelligence.com/regraph/>.
- 7 sdfp. <https://graphviz.org/docs/layouts/sfdp/>.
- 8 Sigma.js: a JavaScript library aimed at visualizing graphs of thousands of nodes and edges. <https://www.sigmapjs.org/>.
- 9 Skewed. <http://mozart.diei.unipg.it/gdcontest/contest2011/composers.xml>.
- 10 TileLayer (@deck.gl/geo-layers). <https://deck.gl/docs/api-reference/geo-layers/tile-layer>. Accessed: 2026.
- 11 yworks. <https://yworks.com/products/yed>.
- 12 James Abello, Frank Van Ham, and Neeraj Krishnan. ASK-GraphView: A large scale graph visualization system. In *IEEE Transactions on Visualization and Computer Graphics*, volume 12, pages 669–676. IEEE, 2006.
- 13 Andrew Beveridge and Michael Chemers. The game of game of thrones: Networked concordances and fractal dramaturgy. In *Reading Contemporary Serial Television Universes*, pages 201–225. Routledge, 2018.
- 14 Prosenjit Bose and J. Mark Keil. On the stretch factor of the constrained Delaunay triangulation. In *3rd International Symposium on Voronoi Diagrams in Science and Engineering (ISVD)*, pages 25–31. IEEE, 2006. doi:10.1109/ISVD.2006.7.
- 15 Ulrik Brandes and Christian Pich. Eigensolver methods for progressive multidimensional scaling of large data. In *Graph Drawing: 14th International Symposium, GD 2006, Karlsruhe, Germany, September 18–20, 2006. Revised Papers 14*, pages 42–53. Springer, 2007.
- 16 Bernard Chazelle. A theorem on polygon cutting with applications. In *23rd Annual Symposium on Foundations of Computer Science (sfcs 1982)*, pages 339–349. IEEE, 1982.
- 17 Boris Delaunay et al. Sur la sphere vide. *Izv. Akad. Nauk SSSR, Otdelenie Matematicheskii i Estestvennyka Nauk*, 7(1):793–800, 1934.
- 18 Douglas Demyen and Michael Buro. Efficient triangulation-based pathfinding. In *Proceedings of the 21st National Conference on Artificial Intelligence (AAAI)*, pages 942–947, 2006.
- 19 David P Dobkin, Emden R Gansner, Eleftherios Koutsofios, and Stephen C North. Implementing a general-purpose edge router. In *Graph Drawing: 5th International Symposium, GD'97 Rome, Italy, September 18–20, 1997 Proceedings 5*, pages 262–271. Springer, 1997.
- 20 Vid Domiter and Borut Žalik. Sweep-line algorithm for constrained delaunay triangulation. *International Journal of Geographical Information Science*, 22(4):449–462, 2008.
- 21 Ran Duan, Jiayi Mao, Xiao Mao, Xinkai Shu, and Longhui Yin. Breaking the sorting barrier for directed single-source shortest paths. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, 2025. <https://arxiv.org/abs/2504.17033>.
- 22 Tim Dwyer, Yehuda Koren, and Kim Marriott. IPSep-CoLa: An incremental procedure for separation constraint layout of graphs. *IEEE Transactions on Visualization and Computer Graphics*, 12(5):821–828, 2006. doi:10.1109/TVCG.2006.156.

- 505 23 Tim Dwyer and Lev Nachmanson. Fast edge-routing for large graphs. In *Graph Drawing: 17th International Symposium, GD 2009, Chicago, IL, USA, September 22-25, 2009. Revised Papers 17*, pages 147–158. Springer, 2010.
- 506
- 507
- 508 24 Max Franz, Christian T. Lopes, Gerardo Huck, Yue Dong, Onur Sumer, and Gary D. Bader. Cytoscape.js: a graph theory library for visualisation and analysis. *Bioinformatics*, 32(2):309–311, 2016.
- 509
- 510
- 511 25 Emden R. Gansner, Yehuda Koren, and Stephen North. Topological fisheye views for visualizing large graphs. *IEEE Transactions on Visualization and Computer Graphics*, 11(4):457–468, 2005.
- 512
- 513
- 514 26 Emden R. Gansner and Stephen C. North. An open graph visualization system and its applications to software engineering. *Software: Practice and Experience*, 30(11):1203–1233, 2000.
- 515
- 516
- 517 27 Leonidas Guibas, John Hershberger, Daniel Leven, Micha Sharir, and Robert E. Tarjan. Linear-time algorithms for visibility and shortest path problems inside triangulated simple polygons. *Algorithmica*, 2(1–4):209–233, 1987.
- 518
- 519
- 520 28 David Harel and Yehuda Koren. A fast multi-scale method for drawing large graphs. In *Journal of Graph Algorithms and Applications*, volume 6, pages 179–202, 2002.
- 521
- 522 29 Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968. doi:10.1109/TSSC.1968.300136.
- 523
- 524
- 525 30 John Hershberger and Jack Snoeyink. Computing minimum length paths of a given homotopy class. *Computational geometry*, 4(2):63–97, 1994.
- 526
- 527 31 John Hershberger and Subhash Suri. An optimal algorithm for Euclidean shortest paths in the plane. *SIAM Journal on Computing*, 28(6):2215–2256, 1999.
- 528
- 529 32 Danny Holten. Hierarchical edge bundles: Visualization of adjacency relations in hierarchical data. *IEEE Transactions on Visualization and Computer Graphics*, 12(5):741–748, 2006.
- 530
- 531 33 Danny Holten and Jarke J. Van Wijk. Force-directed edge bundling for graph visualization. In *Computer Graphics Forum*, volume 28, pages 983–990. Wiley Online Library, 2009.
- 532
- 533 34 Yifan Hu. Efficient, high-quality force-directed graph drawing. *Mathematica Journal*, 10(1):37–71, 2005.
- 534
- 535 35 Christophe Hurter, Ozan Ersoy, and Alexandru Telea. Graph bundling by kernel density estimation. In *Computer graphics forum*, volume 31, pages 865–874. Wiley Online Library, 2012.
- 536
- 537
- 538 36 Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller, James W. Thatcher, and Jean D. Bohlinger, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, New York, 1972.
- 539
- 540
- 541 37 Sebastian Knopp, Peter Sanders, Dominik Schultes, Falk Schieferdecker, and Dorothea Wagner. Computing many-to-many shortest paths using highway hierarchies. In *Proceedings of the 9th Workshop on Algorithm Engineering and Experiments (ALENEX)*, pages 36–45. SIAM, 2007.
- 542
- 543
- 544 38 D. T. Lee and Franco P. Preparata. Euclidean shortest paths in the presence of rectilinear barriers. *Networks*, 14(3):393–410, 1984.
- 545
- 546 39 Jure Leskovec, Jon Kleinberg, and Christos Faloutsos. Graph evolution: Densification and shrinking diameters. *ACM transactions on Knowledge Discovery from Data (TKDD)*, 1(1):2–es, 2007.
- 547
- 548
- 549 40 Lev Nachmanson, Roman Prutkin, Bongshin Lee, Nathalie Henry Riche, Alexander E Holroyd, and Xiaoji Chen. Graphmaps: Browsing large graphs as interactive maps. In *Graph Drawing and Network Visualization: 23rd International Symposium, GD 2015, Los Angeles, CA, USA, September 24-26, 2015, Revised Selected Papers 23*, pages 3–15. Springer, 2015.
- 550
- 551
- 552
- 553 41 Lev Nachmanson, George G. Robertson, and Bongshin Lee. Drawing graphs with GLEE. In *Graph Drawing: 15th International Symposium, GD 2007*, pages 389–394. Springer, 2008.
- 554
- 555 42 Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation ranking: Bringing order to the web. *Stanford InfoLab*, 249(373):1–17, 1999.
- 556

- 557 **43** Alexandre Perrot and David Auber. Cornac: Tackling huge graph visualization with big data
558 infrastructure. *IEEE Transactions on Big Data*, 6(1):80–92, 2018.
- 559 **44** Benedek Rozemberczki and Rik Sarkar. Characteristic Functions on Graphs: Birds of a
560 Feather, from Statistical Descriptors to Parametric Models. In *Proceedings of the 29th ACM
561 International Conference on Information and Knowledge Management (CIKM '20)*, page
562 1325–1334. ACM, 2020.
- 563 **45** Jonathan Richard Shewchuk. Delaunay refinement algorithms for triangular mesh generation.
564 *Computational Geometry*, 22(1–3):21–74, 2002.
- 565 **46** Kozo Sugiyama, Shojiro Tagawa, and Mitsuhiro Toda. Methods for visual understanding
566 of hierarchical system structures. *IEEE Transactions on Systems, Man, and Cybernetics*,
567 11(2):109–125, 1981. doi:10.1109/TSMC.1981.4308636.
- 568 **47** Michael Wybrow, Kim Marriott, and Peter J. Stuckey. Orthogonal connector routing. *Lecture
569 Notes in Computer Science*, 5849:219–231, 2010.